Hochschule für
Wirtschaft und Recht Berlin
Berlin School of Economics and Law

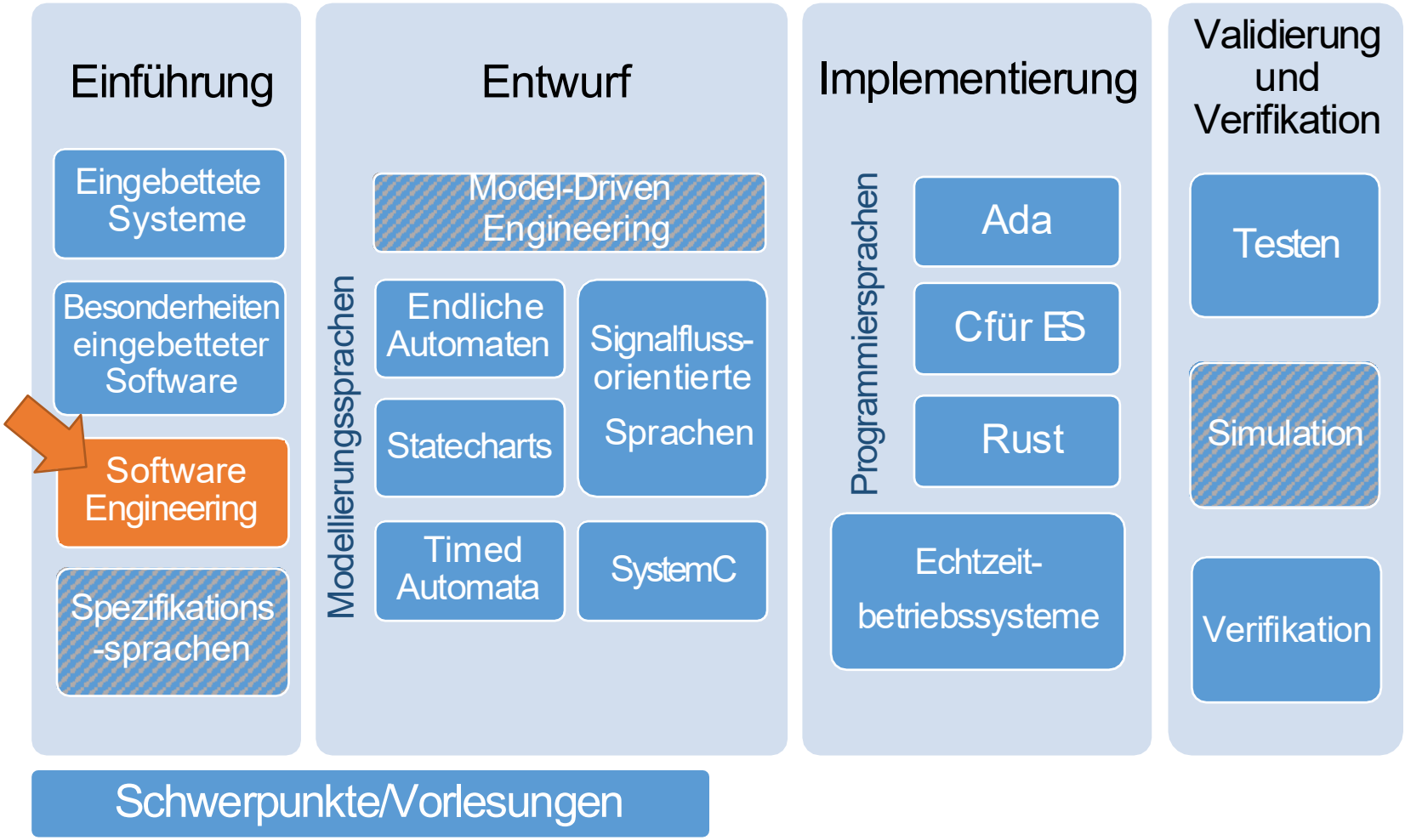
Embedded Systems Engineering

Software Engineering

Prof. Dr.-Ing. Sebastian Schlesinger
Professor of Business Computer Science
Cybersecurity and Embedded Systems Engineering

Diese Vorlesung

Hochschule für Wirtschaft und Recht Berlin
Berlin School of Economics and Law

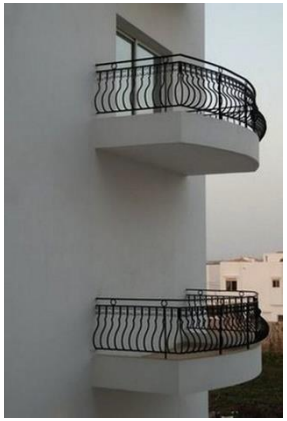


Inhalt

- Einführung
- Softwareentwicklungsprozesse
- Fazit + Literatur

Motivation

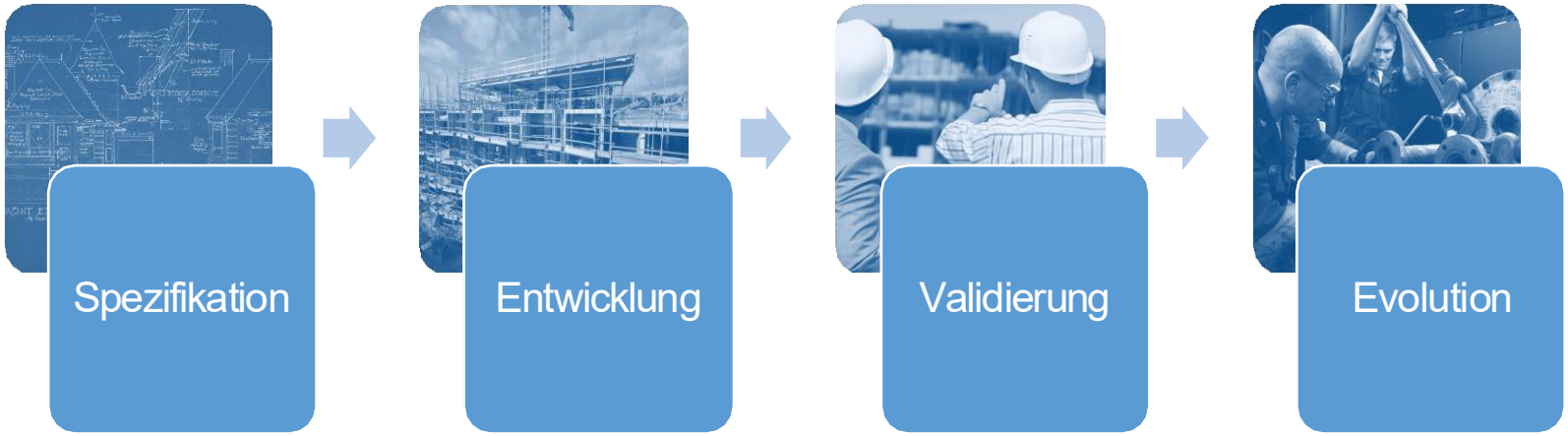
Planlos programmieren ist wie planlos bauen.



Phasen des Software Engineering

Hochschule für Wirtschaft und Recht Berlin
Berlin School of Economics and Law

Stattdessen nähern wir uns der Lösung schrittweise in mehreren Phasen



Software Engineering

Hochschule für Wirtschaft und Recht Berlin
Berlin School of Economics and Law

„Software-Engineering ist eine **technische Disziplin**, die sich mit **allen Aspekten** der Softwareherstellung beschäftigt."
Ian Sommerville

Prozessaktivitäten:

- Spezifikation, Entwicklung, Validierung, Evolution (Weiterentwicklung)

Softwareentwicklungsprozesse

- Reihenfolge und Häufigkeit der Prozessaktivitäten

Methoden und Werkzeuge des Softwareentwicklung

- Anforderungsmanagement, Spezifikation- und Modellierungssprachen
- Testen, Simulierung, Model Checking

Inhalt

- Einführung
- Softwareentwicklungsprozesse
- Fazit + Literatur

Softwareentwicklungsprozesse

Hochschule für Wirtschaft und Recht Berlin
Berlin School of Economics and Law

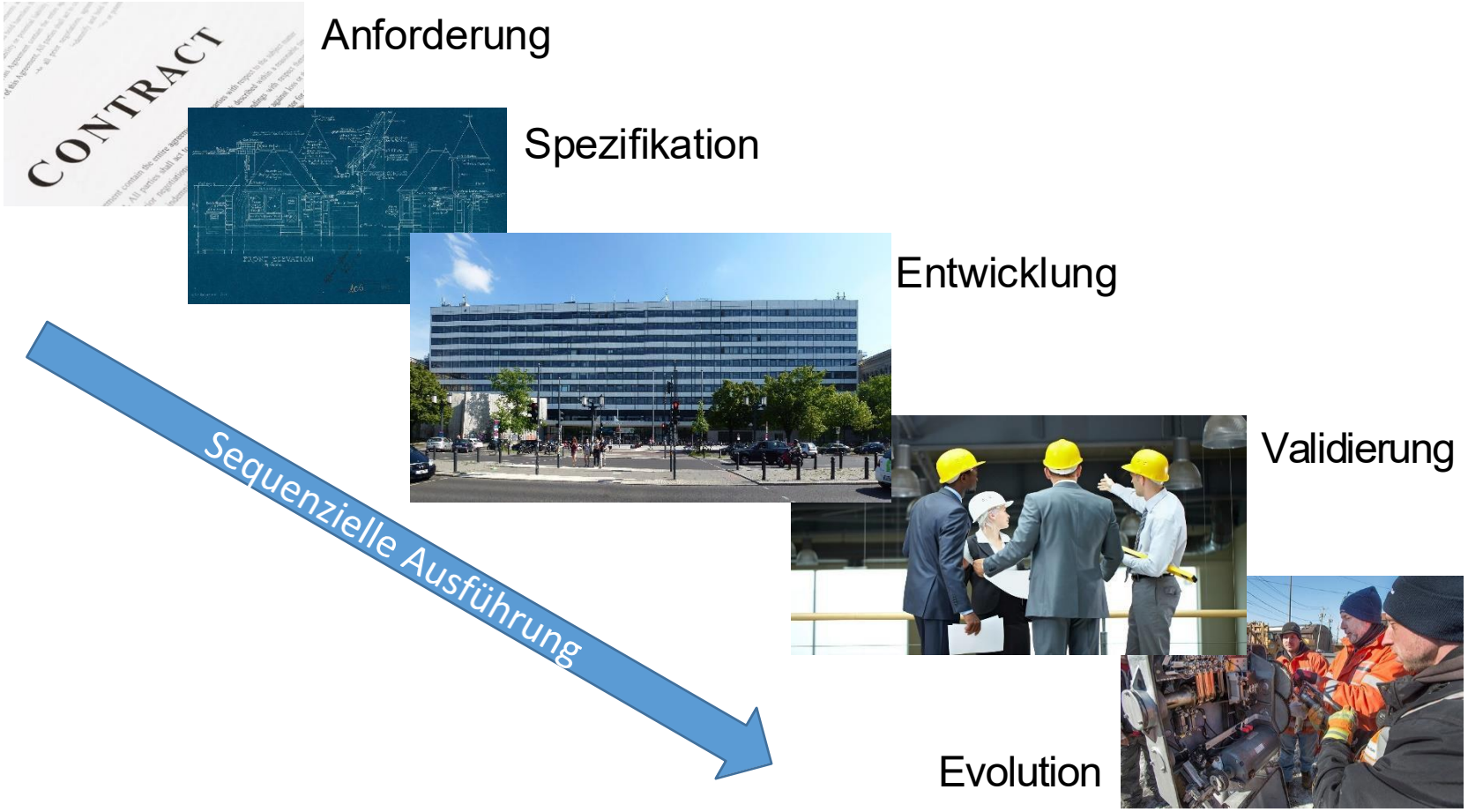
Ausgehend von informellen Anforderungen werden die **grundlegenden Aktivitäten** in einem **Softwareentwicklungsprozess** kombiniert.

Spezifikation	Entwicklung	Validierung	Evolution
• Definition der zu entwickelnden Software und ihrer Einsatzbedingungen	• Entwurf und Programmierung der Software	• Überprüfung der Software auf Erfüllung der Anforderungen	• Weiterentwicklung und Anpassung an geänderte Anforderungen

Verschiedene Softwareentwicklungsprozesse legen den **Schwerpunkt** auf verschiedene Aspekte der Softwareentwicklung.

Es gibt keinen idealen Prozess für alle Situationen!

Beispiel


CONTRACT
Anforderung
Spezifikation
Entwicklung
Validierung
Evolution
Sequenzielle Ausführung

Das Wasserfallmodell

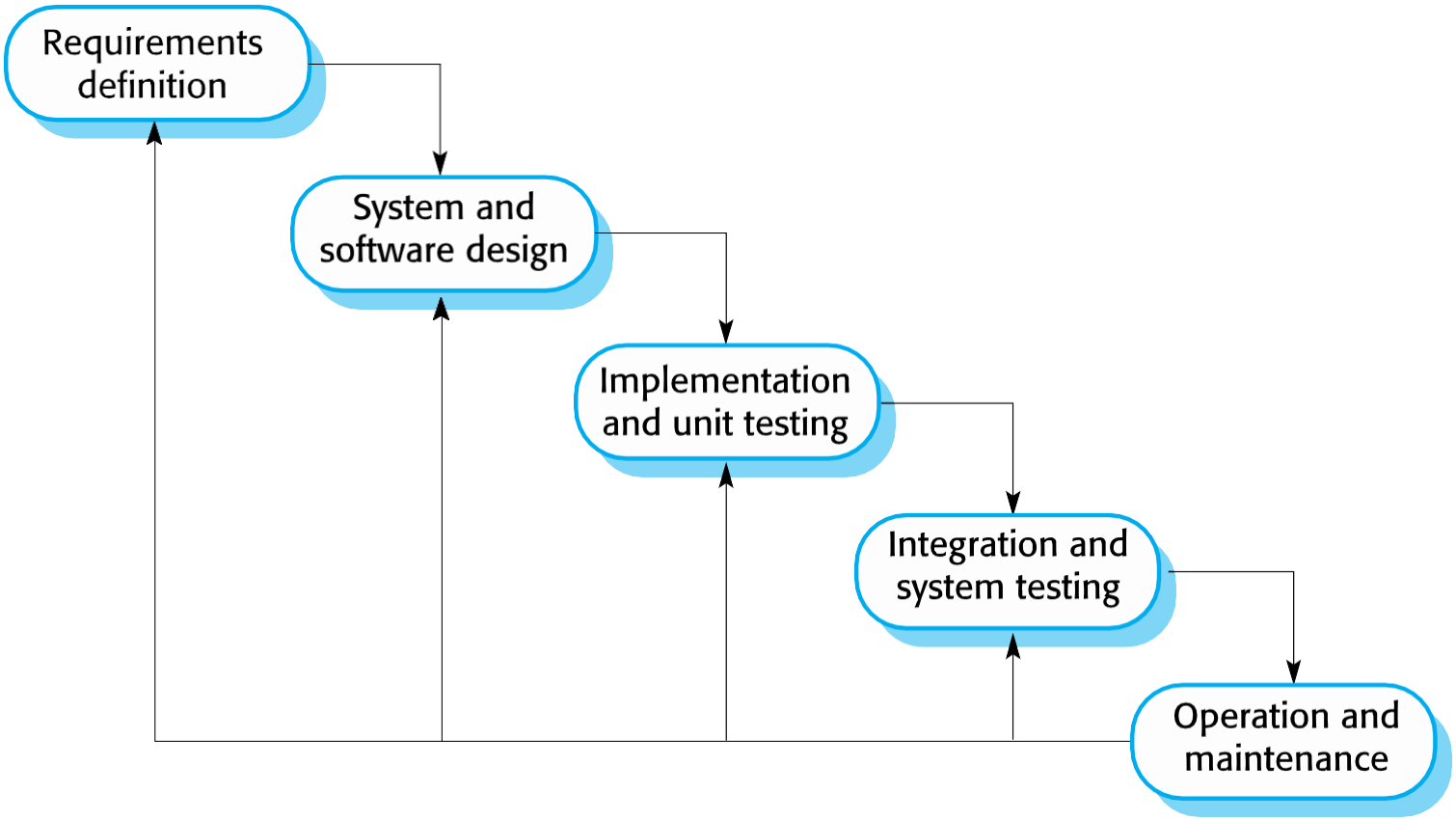
Aufteilung des Entwicklungsprozesses in feste, **sequentielle** Phasen

- *Vorteile*
  Klare Abgrenzung der einzelnen Phasen…
  …ermöglicht einfache Koordinierung der Arbeitsschritte

- *Nachteile*
  Umfangreiche Planung nötig
  Auf **Veränderungen** kann während des Prozesses schlecht reagiert werden…
  …darum müssen die Anforderungen weitestgehend stabil sein
  Fehler werden erst sehr spät erkannt (späte Integration und Systemtests)

Geeignet für stabile Großprojekte mit mehreren Entwicklungsteams

Das Wasserfallmodell

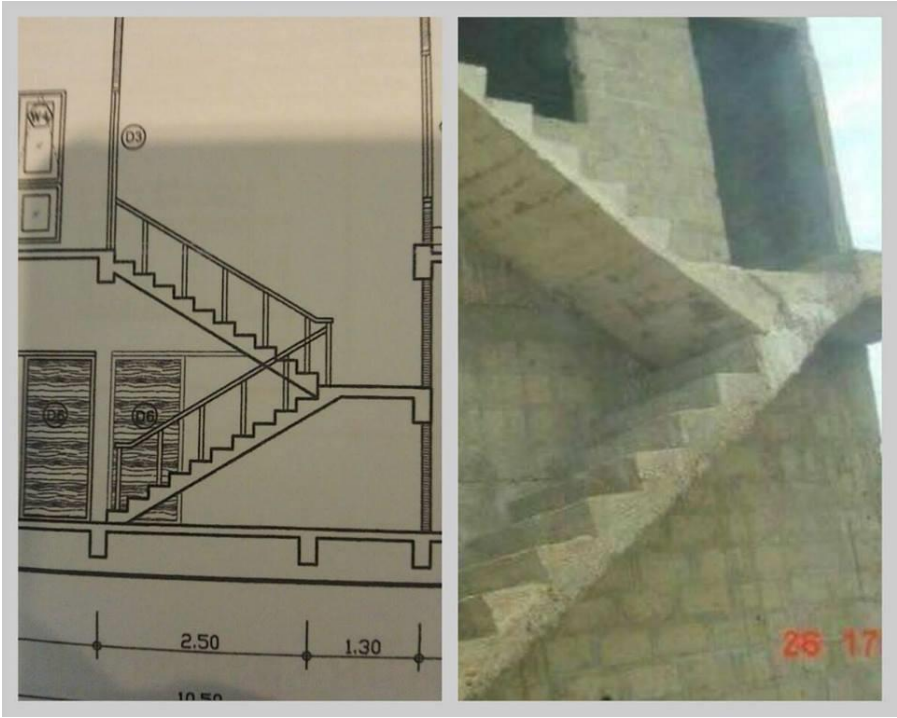


Ian Sommerville, Software-Engineering, Chapter 2

Validierung

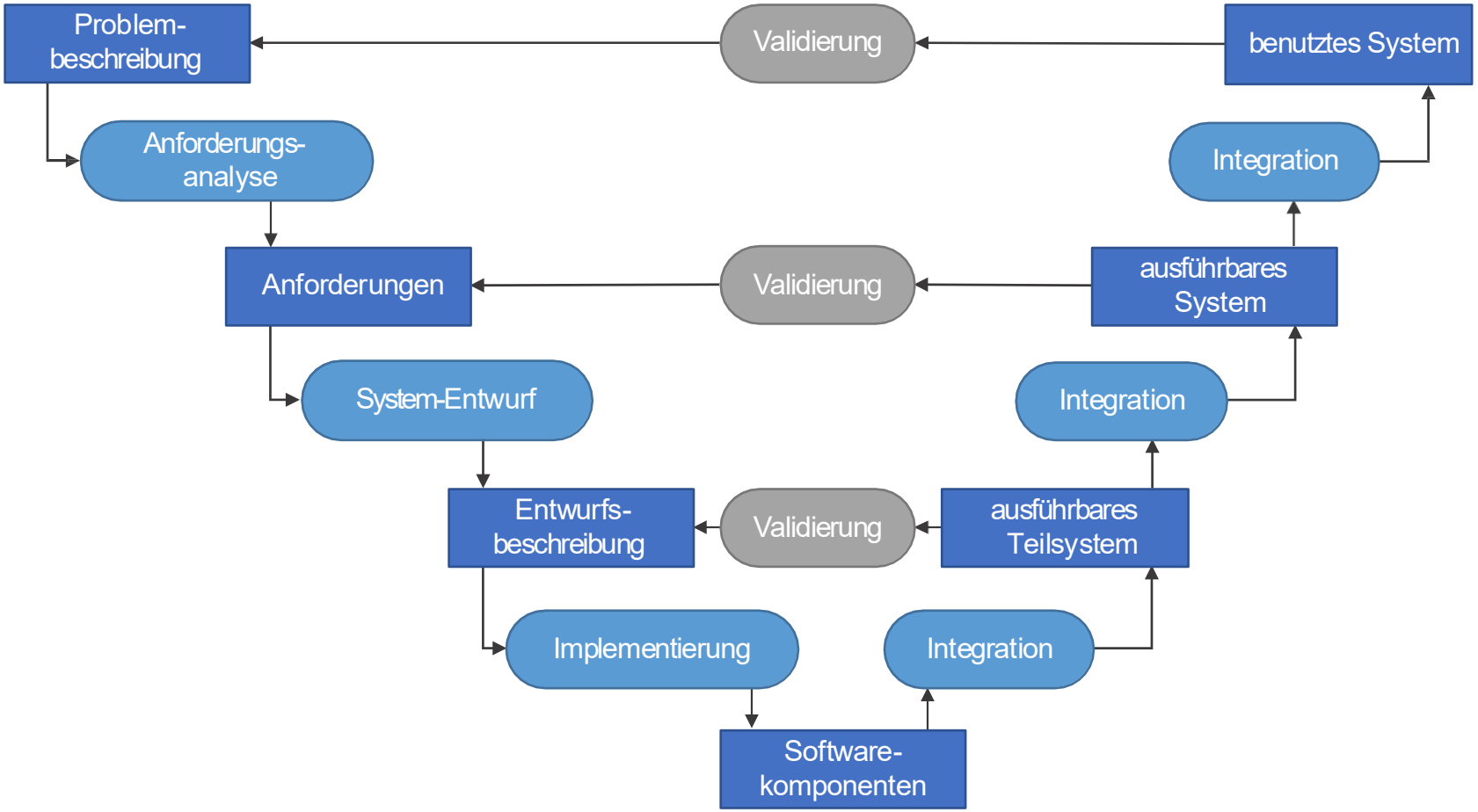
Hochschule für Wirtschaft und Recht Berlin
Berlin School of Economics and Law

Alles gut geplant, aber nur Quatsch implementiert…



…was nun?

Das V-Modell


Problem-
beschreibung
Validierung
benutztes System
Anforderungs-
analyse
Integration
Anforderungen
Validierung
ausführbares
System
System-Entwurf
Integration
Entwurfs-
beschreibung
Validierung
ausführbares
Teilsystem
Implementierung
Integration
Software-
komponenten

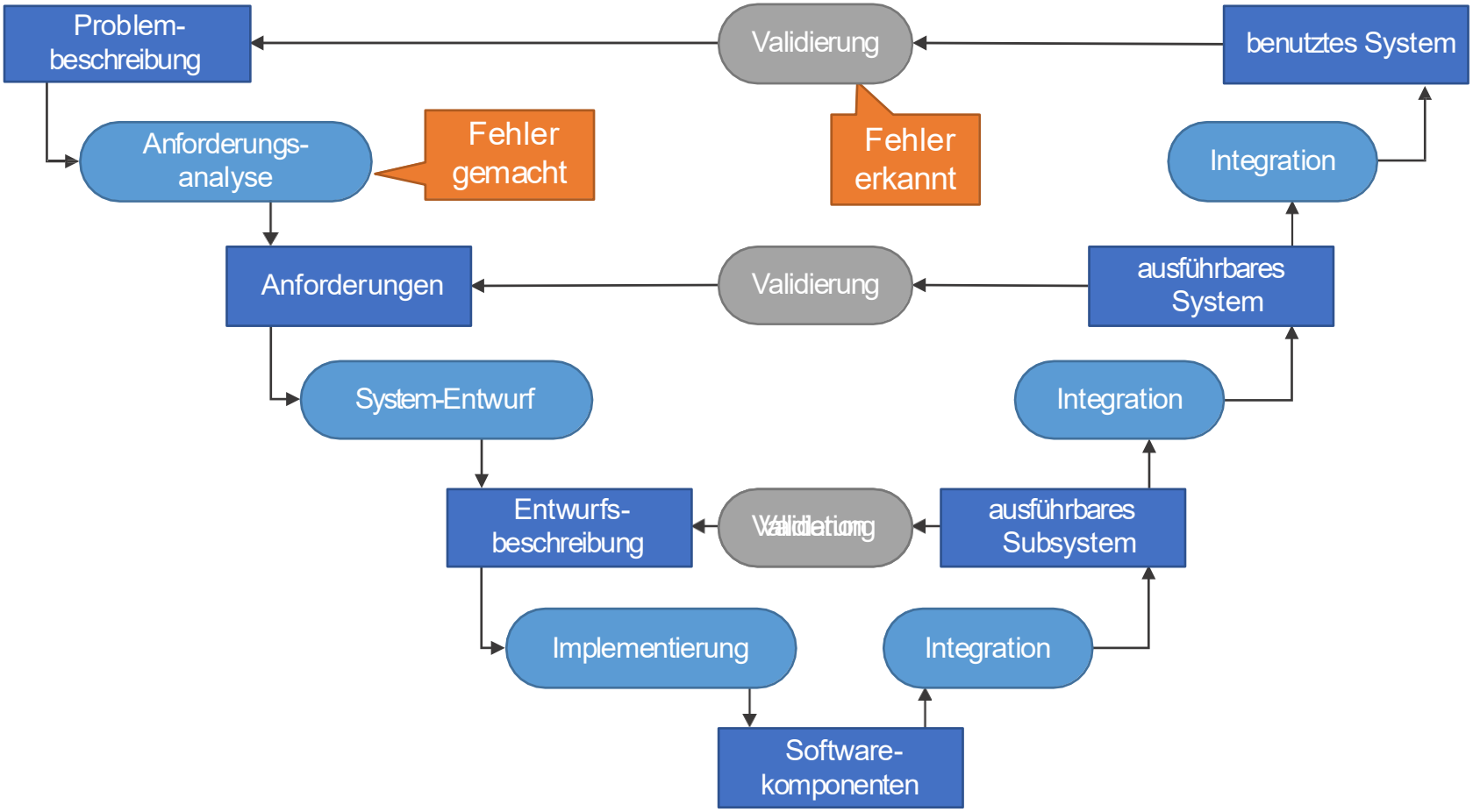
Das V-Modell

Erweitert das Wasserfallmodell um vorher geplante **Testaktivitäten**

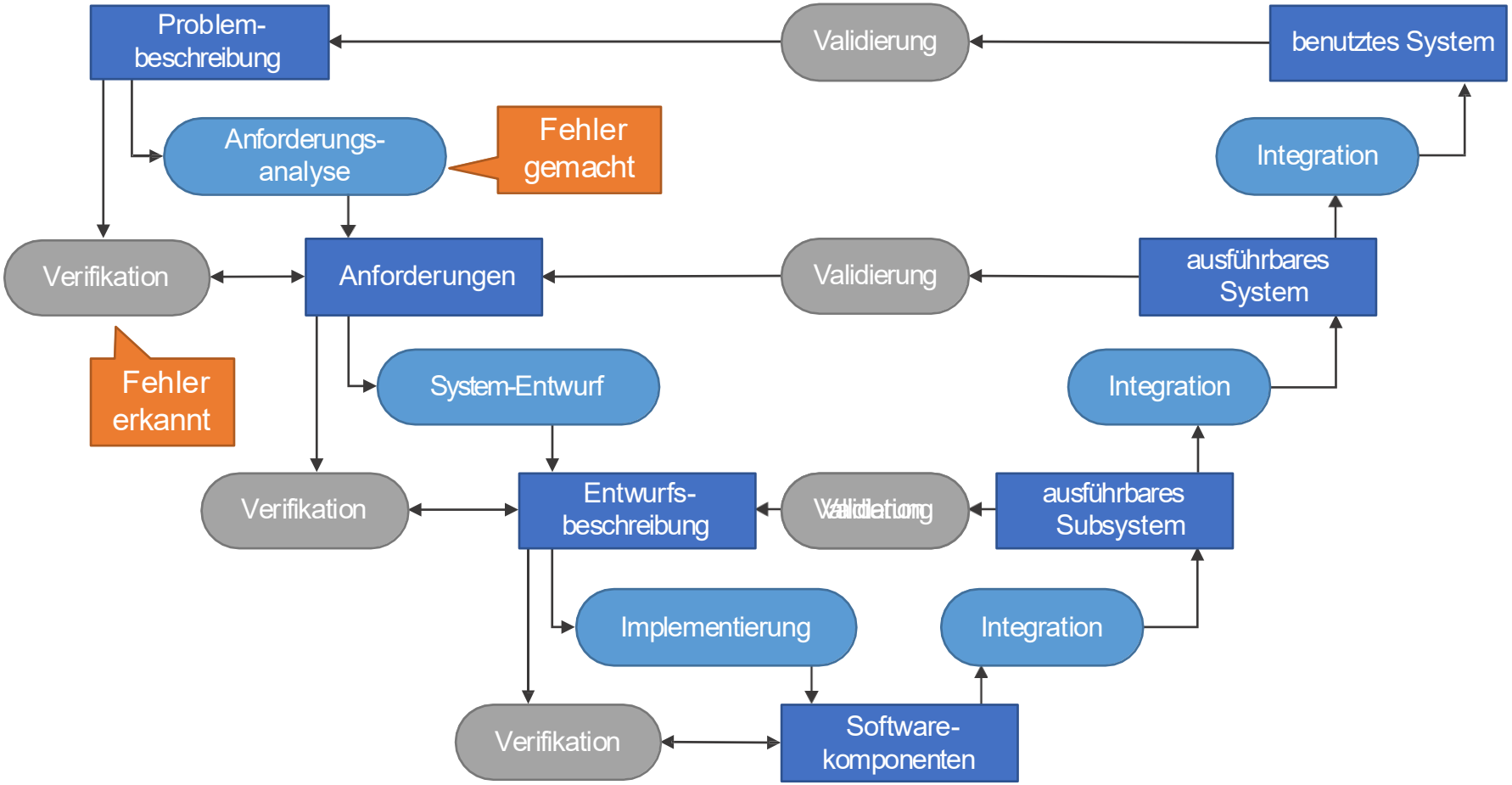
- Stärkerer Fokus auf der **Validierungsphase**

- *Entwickler-/Komponententests*
  Komponenten werden unabhängig von anderen Komponenten getestet
  Oftmals durch den Entwickler selbst, um Programmierfehler zu finden
- *Systemtest*
  Das System wird vollständig integriert getestet, eventuell mehrstufig
  Dient dem Auffinden von Schnittstellenproblemen und Fehlern in der Interaktion von Komponenten
- *Abnahmetest (Alpha-Test)*
  Testen des Gesamtsystems mit vom Kunden zur Verfügung gestellten Testdaten
  Dient der direkten Validierung mit dem Kunden

Das V-Modell ohne Verifikation



…und mit Verifikation



Verifikation und Validierung

- **Validierung**
  - Passt das implementierte System zum Plan und zur Spezifikation?
- **Verifikation**
  - Passt der Plan zur Spezifikation auf der nächsthöheren Abstraktionsebene?
- **Mathematisch fundierte Spezifikations- und Modellierungssprachen** ermöglichen eine umfassende Verifikation

Dynamische Anforderungen

Hochschule für Wirtschaft und Recht Berlin
Berlin School of Economics and Law

Manchmal ändern sich die Anforderungen im Laufe eines Projektes.

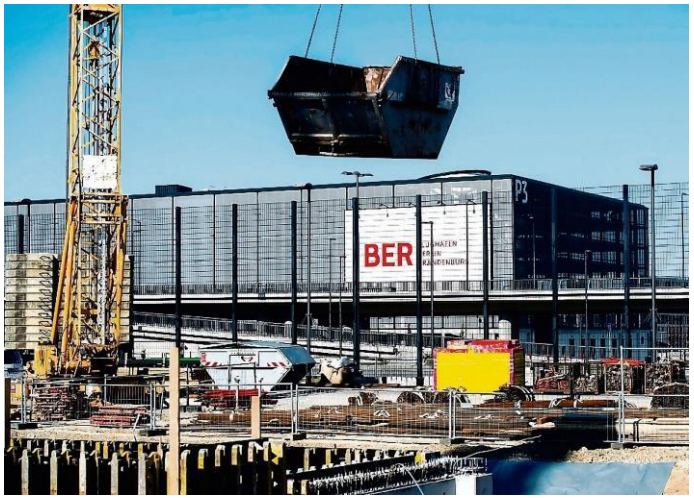


Foto: Tagesspiegel

Wie kann man damit umgehen?

Inkrementelle Entwicklung

Alternativ/komplementär zur Verifikation

Inkrementelles Entwicklungsmodell mit frühen Prototypen

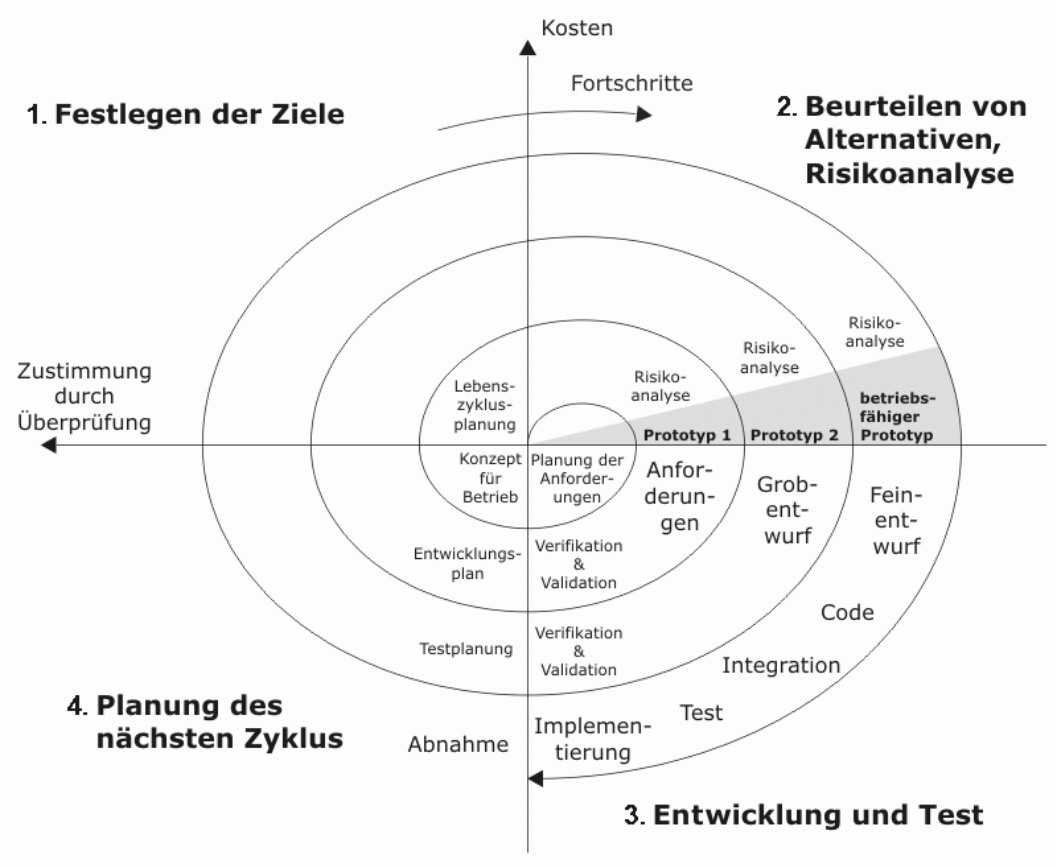
- Besonderer Fokus auf die **Transparenz** des Entwicklungsprozesses

Die Entwicklung wird durch eine **Risikobewertung** gesteuert

- In jeder Iteration werden die Risiken des Projekts **neu bewertet**
- Kann ein Risiko nicht **beseitigt** werden, gilt das Projekt als gescheitert

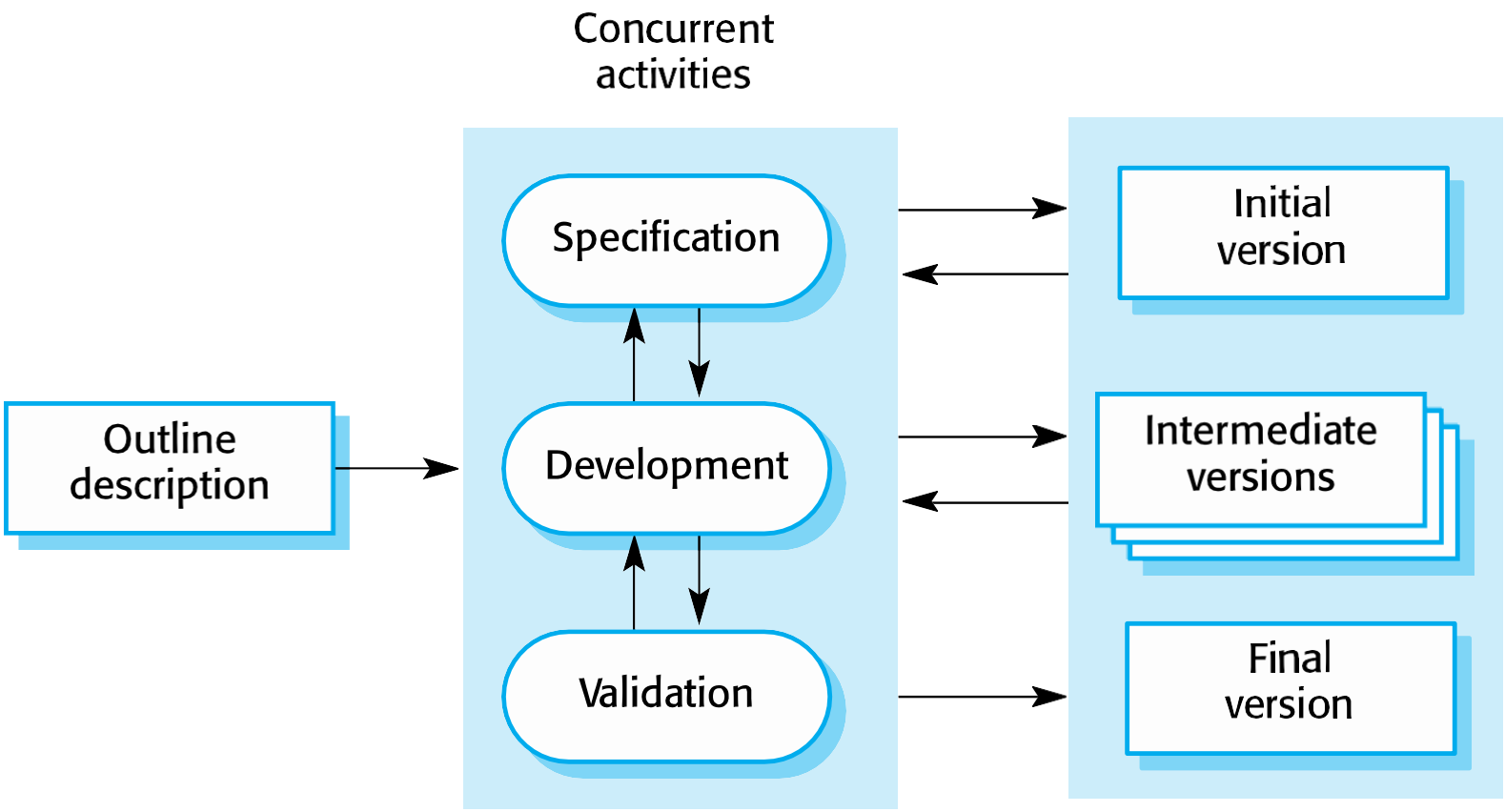
- *Vorteile*
  Iterative Entwicklung reduziert Risiko
  Frühes Testen (auch durch Benutzer) von Prototypen
- *Nachteile*
  Eher für große Projekte geeignet
  Risikoanalyse kann mitunter sehr teuer sein
  Prototypen können besonders im eingebetteten Bereich teuer sein

Spiralmodell nach Boehm



Erstellt von Conny aus der deutschsprachigen Wikipedia, CC BY-SA 3.0

Inkrementelle Entwicklung



Ian Sommerville, Software-Engineering, Chapter 2

Inkrementelle Entwicklung

Spezifikation, Entwicklung und Validierung werden **parallel** ausgeführt

- Zwischen den einzelnen Aktivitäten werden Rückmeldungen ausgetauscht, um Fehler früh zu beheben

Durch frühe Prototypen (Anfangsimplementierung) können Kundenwünsche während der Entwicklung berücksichtigt werden

- Bis zur finalen Version werden mehrere Zwischenversionen erstellt

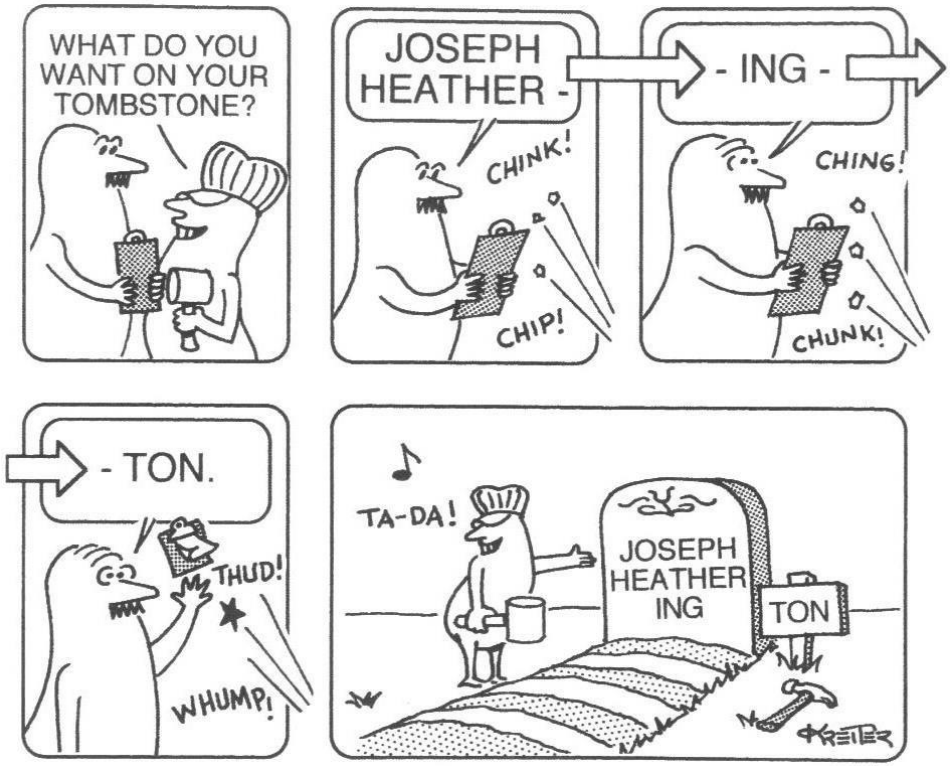
- *Probleme der Inkrementellen Entwicklung*

  Bürokratische Umgebungen harmonieren nicht gut mit informelleren Prozessen (z.B. sicherheitskritische Software)

  Der Gesamtfortschritt ist nicht anhand eines starren Planes messbar

Inkrementelle Entwicklung ist das Fundament **agiler Entwicklung**!

Wozu agile Entwicklungsprozesse?



Agile Entwicklungsprozesse ermöglichen **veränderliche Anforderungen**

Agile Entwicklungsprozesse

Agile Entwicklungsprozesse sind inkrementelle Entwicklungsprozesse mit **sehr kleinen Inkrementen** und **vielen Prototypen**.

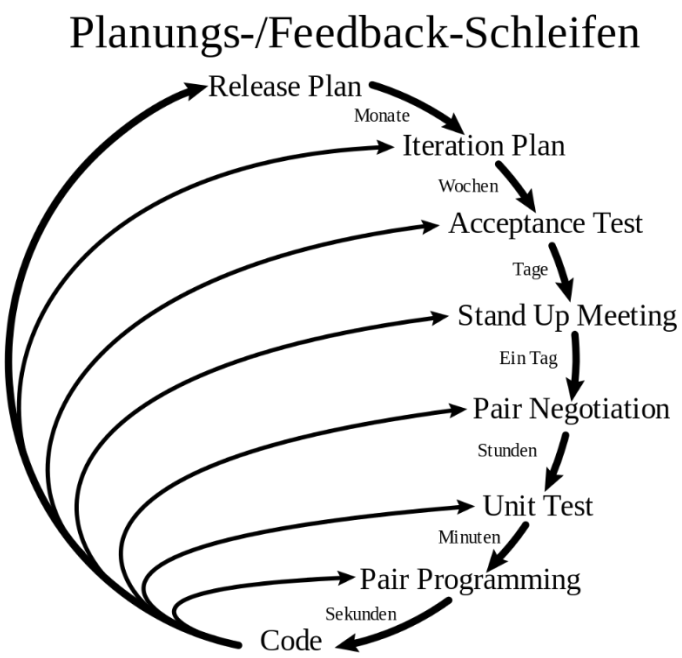
- Weit verbreitet bei der Entwicklung **klassischer Software**
- *Grundlegende Prinzipien*
  Einbeziehen des Kunden, inkrementelle Auslieferung
  Gezielt Fähigkeiten des Teams nutzen („Menschen statt Prozesse“)
  Offen für Änderungen und Einfachheit des Prozesses und Produkts

“**What's wrong** with the analogy between software and building construction?”
Frage von oscarkuo auf StackOverflow.com

„[Architect's clients] can't take delivery of your **finished** underground carpark and ask you **to add** an airport (also underground)”
Antwort von Martin Beckett

Direktes Feedback

Annahme: der Kunde kennt die Anforderungen selbst noch nicht genau



Quelle: Wikipedia

Entwicklungsstile

Hochschule für Wirtschaft und Recht Berlin
Berlin School of Economics and Law

Oft wird zwischen **plangesteuerten** und **agilen** Prozessen unterschieden:

- *Plangesteuerte Entwicklungsprozesse*
  - Die Abfolge der Aktivitäten und deren Ziele **im Voraus** fest eingeplant
  - Der tatsächliche Fortschritt lässt sich leicht anhand **des Planes** feststellen
  - Eignen sich am besten für große Softwareprojekte mit **festen Anforderungen**

- *Agile Entwicklungsprozesse*
  - Spezifikation und Entwicklung können sich **überschneiden**
  - Die Software wird **inkrementell** entwickelt
  - Frühe Prototypen ermöglichen direktes **Feedback** vom Kunden
  - Eignen sich gut für Umgebungen mit **wechselnden Anforderungen**

Plangesteuert oder agil?

Hochschule für Wirtschaft und Recht Berlin
Berlin School of Economics and Law

Wo empfiehlt sich ein plangesteuerter, wo ein agiler Prozess?

Aussage	Empfehlung
Die Entwicklung benötigt eine sehr detaillierte Spezifikation.	
Inkrementelle Auslieferung ist realistisch und schnelles Kundenfeedback gewünscht.	
Das Entwicklerteam ist zu groß für informellen Austausch.	
Die Anforderungen sind komplex und haben hohen Analysebedarf (z.B. Echtzeitsystem).	
Das System ist sehr langlebig, umfangreiche Dokumentation erwünscht.	
Es sind sehr gute Werkzeuge zur nachvollziehbaren Entwicklung des Entwurfs vorhanden.	
Kunde verlangt belastbare Spezifikation vorab (Pflichtenheft).	
Das Team ist dezentral organisiert, Schnittstellen müssen gründlich entworfen werden.	
Das Team verfügt über viele erfahrene Programmierer die selbstständig arbeiten können.	
Das System unterliegt externen Vorschriften (Genehmigung nach Standard).	

Plangesteuert oder agil?

Hochschule für Wirtschaft und Recht Berlin
Berlin School of Economics and Law

Wo empfiehlt sich ein plangesteuerter, wo ein agiler Prozess?

Aussage	Empfehlung
Die Entwicklung benötigt eine sehr detaillierte Spezifikation.	Plangesteuert
Inkrementelle Auslieferung ist realistisch und schnelles Kundenfeedback gewünscht.	Agil
Das Entwicklerteam ist zu groß für informellen Austausch.	Plangesteuert
Die Anforderungen sind komplex und haben hohen Analysebedarf (z.B. Echtzeitsystem).	Plangesteuert
Das System ist sehr langlebig, umfangreiche Dokumentation erwünscht.	Plangesteuert
Es sind sehr gute Werkzeuge zur nachvollziehbaren Entwicklung des Entwurfs vorhanden.	Agil
Kunde verlangt belastbare Spezifikation vorab (Pflichtenheft).	Plangesteuert
Das Team ist dezentral organisiert, Schnittstellen müssen gründlich entworfen werden.	Plangesteuert
Das Team verfügt über viele erfahrene Programmierer die selbstständig arbeiten können.	Agil
Das System unterliegt externen Vorschriften (Genehmigung nach Standard).	Plangesteuert

Für eingebettete Systeme sind generell **plangesteuerte Prozesse** zu empfehlen.

Wiederverwendungsorientierte Prozesse

Während der Entwicklung entstehen zunächst **versunkene Kosten**

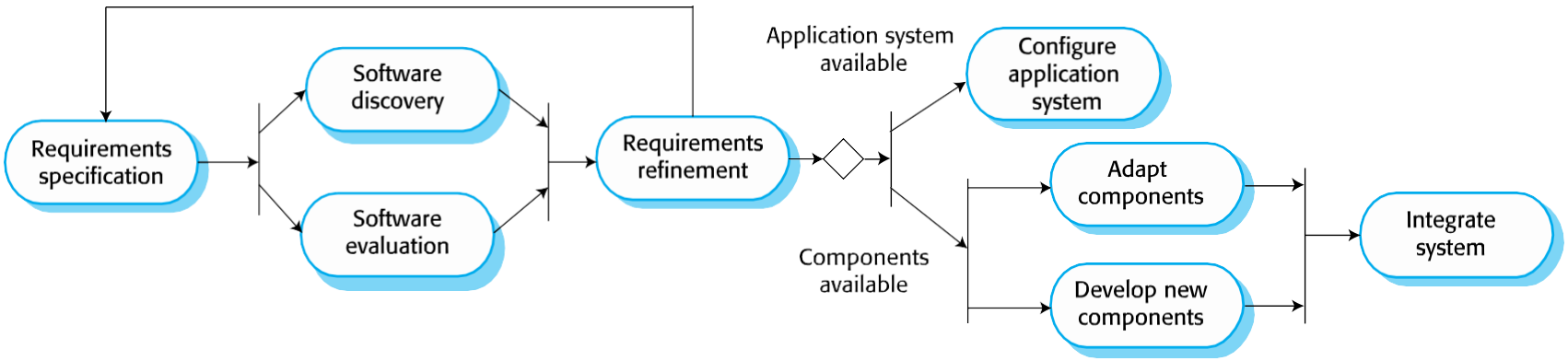
- Bis zur **Fertigstellung** ist die Software größtenteils wertlos

Die Entwicklung eines Systems kann beschleunigt (und damit günstiger) werden, indem Komponenten **wiederverwendet** werden.

- „Time to Market“ verkürzt, Entwicklungskosten reduziert

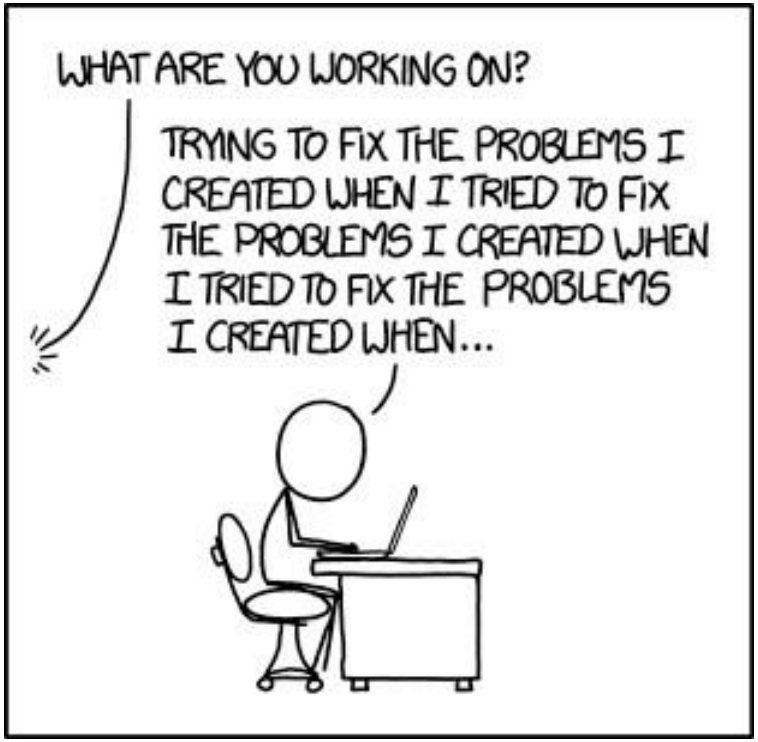
- *Analyse der Komponenten*
  Aus den bestehenden Komponenten werden zur Anforderungsspezifikation passende ausgesucht
- *Anpassung der Anforderungen*
  Eventuell werden die Anforderungen bestehender Komponenten angepasst
- *Ergänzung und Integration der Komponenten*
  Fehlende Komponenten werden neu erstellt und zusammen mit den angepassten Komponenten zum Gesamtsystem integriert

Wiederverwendungsorientierte Prozesse



Ian Sommerville, Software-Engineering, Chapter 2

Evolution und Wiederverwendung

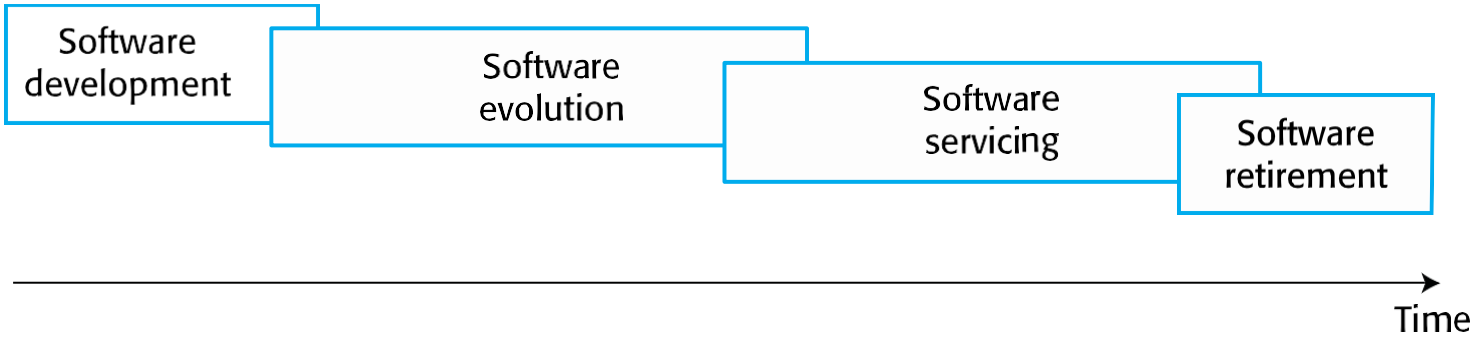


Evolution und Wiederverwendung

Hochschule für Wirtschaft und Recht Berlin
Berlin School of Economics and Law

Ein Großteil der Softwarekosten werden bei der **Evolution** verursacht:

Änderung im Anforderungsprofil, aufgetretene Fehler, nachlassende Leistung, neue Hardwarebedingungen etc. erzwingen nachträgliche ***Anpassungen*** *und* ***Verbesserungen*** *der Software oder einzelner Komponenten*



Während der späteren Instandhaltungsphase werden nur noch kleine, unerlässliche Änderungen vorgenommen.

Ian Sommerville, Software-Engineering, Chapter 9

Fazit

- Moderne Software muss systematisch entwickelt werden
- Ein **Softwareentwicklungsprozess** definiert und strukturiert die grundlegenden Aktivitäten der Softwareentwicklung
- Es gibt **viele** verschiedene Prozesse mit unterschiedlichen **Stärken** und **Schwächen**:
  - **Agile Prozesse** nutzen kleine Inkremente mit vielen Prototypen. Sie eignen sich gut für Umgebungen mit wechselnden Anforderungen.
  - Für eingebettete Systeme sind **plangesteuerte Prozesse** zu empfehlen. Sie eignen sich gut für große Projekte mit festen Anforderungen.
- **Validierung und Verifikation** stellen sicher, dass wir das richtige Produkt richtig entwickeln
- **Wiederverwendung** kann die Entwicklung beschleunigen und Kosten senken

Literatur

- P. LIGGESMEYER & D. ROMBACH: *Software Engineering eingebetteter Systeme*. Spektrum 2005.
- IAN SOMMERVILLE: *Software-Engineering.* Pearson, 2012, 9. Aufl.